

Mastère Cloud Computing et Développement des Applications Distribuées

Module : Administration

Rapport de Projet

Mise en place d'une chaîne CI/CD sécurisée

Réalisé par :

Étudiant : GRENA Hamza

Encadré par :

MAGROUN Hedi

Année universitaire : 2025–2026

Table des matières

Introduction générale	3
1 Présentation du projet CI/CD sécurisé	4
1.1 Objectif du projet	4
1.2 Outils utilisés	4
1.3 Répartition globale du travail	4
2 Semaine 1 : Installation et préparation de l'environnement	6
2.1 Introduction	6
2.2 Travail réalisé par l'Étudiant A	6
2.2.1 Extraction du projet Docker Compose	6
2.2.2 Lancement des conteneurs Docker	7
2.2.3 Vérification de l'état du dépôt Git	7
2.3 Travail réalisé par l'Étudiant B	8
2.3.1 Accès à l'interface Jenkins	8
2.3.2 Récupération du mot de passe initial Jenkins	8
2.3.3 Installation des plugins Jenkins	9
2.4 Conclusion de la semaine 1	9
3 Semaine 2 : Intégration Jenkins, SonarQube et Nexus	10
3.1 Introduction	10
3.2 Travail réalisé par l'Étudiant A	10
3.2.1 Accès à SonarQube	10
3.2.2 Affichage du tableau de bord SonarQube	11
3.2.3 Création du token Jenkins dans SonarQube	11
3.2.4 Ajout du token SonarQube dans Jenkins	12
3.2.5 Configuration de SonarQube dans Jenkins	12
3.2.6 Création du fichier Jenkinsfile	13
3.3 Travail réalisé par l'Étudiant B	13
3.3.1 Accès à Nexus Repository Manager	13
3.3.2 Récupération du mot de passe Nexus	14

3.3.3	Configuration des identifiants Nexus	14
3.3.4	Consultation des repositories Nexus	15
3.3.5	Création d'un repository Maven	15
3.4	Conclusion de la semaine 2	16
4	Exécution du pipeline CI/CD	17
4.1	Introduction	17
4.2	Lancement du pipeline Jenkins	17
4.3	Construction du projet avec Maven	18
4.4	Succès de l'exécution du pipeline	18
4.5	Publication de l'artefact dans Nexus	18
4.6	Résultat obtenu	19
5	Bilan du travail réalisé	20
5.1	Synthèse des tâches	20
5.2	Difficultés rencontrées	21
5.3	Apports du projet	21
	Conclusion générale	22

Introduction générale

Dans le cadre de ce projet, nous avons réalisé une chaîne d'intégration et de livraison continue sécurisée basée sur plusieurs outils DevOps. L'objectif principal est d'automatiser les étapes de construction, d'analyse de qualité du code, de gestion des artefacts et de suivi du projet à travers un pipeline CI/CD.

La solution mise en place repose sur Jenkins comme serveur d'automatisation, SonarQube pour l'analyse statique du code source, Nexus Repository Manager pour la gestion des artefacts, Gitea pour la gestion du code source, Docker pour la conteneurisation et Maven pour la construction du projet.

Le travail a été organisé sur deux semaines et réparti entre deux étudiants. La première semaine a été consacrée à la préparation de l'environnement et à la configuration initiale des outils. La deuxième semaine a été dédiée à l'intégration entre Jenkins, SonarQube et Nexus, ainsi qu'à l'exécution du pipeline CI/CD.

Chapitre 1

Présentation du projet CI/CD sécurisé

1.1 Objectif du projet

L'objectif de ce projet est de mettre en place une chaîne CI/CD sécurisée permettant d'automatiser les différentes étapes du cycle de développement logiciel. Cette chaîne permet de récupérer le code source, compiler le projet, analyser la qualité du code, détecter les vulnérabilités et gérer les artefacts générés.

1.2 Outils utilisés

Les principaux outils utilisés dans ce projet sont :

- **Docker** : utilisé pour lancer les différents services dans des conteneurs.
- **Jenkins** : utilisé pour créer et exécuter le pipeline CI/CD.
- **SonarQube** : utilisé pour analyser la qualité du code source.
- **Nexus Repository Manager** : utilisé pour stocker et gérer les artefacts Maven.
- **Gitea** : utilisé comme plateforme de gestion du code source.
- **Maven** : utilisé pour compiler le projet et générer les artefacts.
- **Git** : utilisé pour le suivi des versions du projet.

1.3 Répartition globale du travail

Le travail a été réparti entre deux étudiants afin d'assurer une meilleure organisation des tâches.

Période	Étudiant A	Étudiant B
Semaine 1	Préparation de l'environnement Docker, extraction du projet, lancement des conteneurs et vérification du dépôt Git.	Configuration initiale de Jenkins, récupération du mot de passe et installation des plugins nécessaires.
Semaine 2	Configuration de SonarQube avec Jenkins, création du token, ajout du token dans Jenkins et préparation du Jenkinsfile.	Configuration de Nexus, création des identifiants, création du repository Maven et test du pipeline CI/CD.

TABLE 1.1 – Répartition globale du travail entre les deux étudiants

Chapitre 2

Semaine 1 : Installation et préparation de l'environnement

2.1 Introduction

La première semaine du projet a été consacrée à la préparation de l'environnement de travail. Cette phase comprend l'extraction du projet Docker Compose, le lancement des conteneurs, l'accès aux interfaces web des outils et la configuration initiale de Jenkins.

2.2 Travail réalisé par l'Étudiant A

2.2.1 Extraction du projet Docker Compose

Cette étape consiste à extraire le projet contenant les fichiers nécessaires au déploiement des services. Le fichier `docker-compose.yml` permet de définir et de lancer les différents conteneurs utilisés dans la chaîne CI/CD, notamment Jenkins, SonarQube, Nexus et Gitea.

```
GNU nano 7.2                                docker-compose.yml *
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    user: root
    ports:
      - "8080:8080"
      - "50000:50000"
    networks:
      - cicd-network
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    networks:

```

FIGURE 2.1 – Extraction du code Docker Compose

2.2.2 Lancement des conteneurs Docker

Après l'extraction du projet, les services ont été lancés à l'aide de Docker Compose. Cette étape permet de démarrer automatiquement les conteneurs nécessaires au fonctionnement de l'environnement CI/CD.

```
useradm@devops:~/cicd-secure-project$ docker compose up -d
[+] up 18/35
* Image sonatype/nexus3 [#####] 10.49MB / 493.1MB Pulling 74.1s
* Image jenkins/jenkins:lts [#####] 15.73MB / 288.7MB Pulling 74.1s
* Image sonarqube:lts-community [#####] 15.73MB / 397.5MB Pulling 74.1s
```

FIGURE 2.2 – Lancement des conteneurs Docker

2.2.3 Vérification de l'état du dépôt Git

L'utilisation de la commande `git status` permet de vérifier l'état du dépôt local. Cette étape est importante pour identifier les fichiers modifiés, ajoutés ou non suivis avant de les valider dans le dépôt Git.

```
useradm@devops:~/cicd-secure-project/app$ git remote -v
origin https://github.com/Hamza-Grena/cicd-secure-project.git (fetch)
origin https://github.com/Hamza-Grena/cicd-secure-project.git (push)
useradm@devops:~/cicd-secure-project/app$ git status
Sur la branche main
rien à valider, la copie de travail est propre
useradm@devops:~/cicd-secure-project/app$
```

FIGURE 2.3 – Vérification de l'état du dépôt Git

2.3 Travail réalisé par l'Étudiant B

2.3.1 Accès à l'interface Jenkins

Après le démarrage des conteneurs, Jenkins est accessible depuis le navigateur à travers son interface web. Cette étape confirme que le service Jenkins fonctionne correctement et qu'il est prêt pour la configuration initiale.



FIGURE 2.4 – Page d'accueil de Jenkins

2.3.2 Récupération du mot de passe initial Jenkins

Lors de la première utilisation de Jenkins, un mot de passe administrateur initial est demandé. Ce mot de passe est récupéré depuis les logs du conteneur ou depuis le fichier généré par Jenkins à l'intérieur du conteneur.

```
valid_lft forever preferred_lft forever
9: veth46f3910@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
link/ether aa:2c:83:d9:57:6f brd ff:ff:ff:ff:ff:ff link-netnsid 1
inet6 fe80::4455:8aff:fe8d:e7a1/64 scope link
valid_lft forever preferred_lft forever
useradm@devops:~/cicd-secure-project$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
cb725dc602f44c29b95b291664b40049
useradm@devops:~/cicd-secure-project$
```

FIGURE 2.5 – Récupération du mot de passe initial de Jenkins

2.3.3 Installation des plugins Jenkins

Les plugins Jenkins sont indispensables pour ajouter les fonctionnalités nécessaires au pipeline. Dans ce projet, les plugins liés à Git, Maven, SonarQube et aux pipelines ont été installés afin de permettre l'automatisation complète de la chaîne CI/CD.

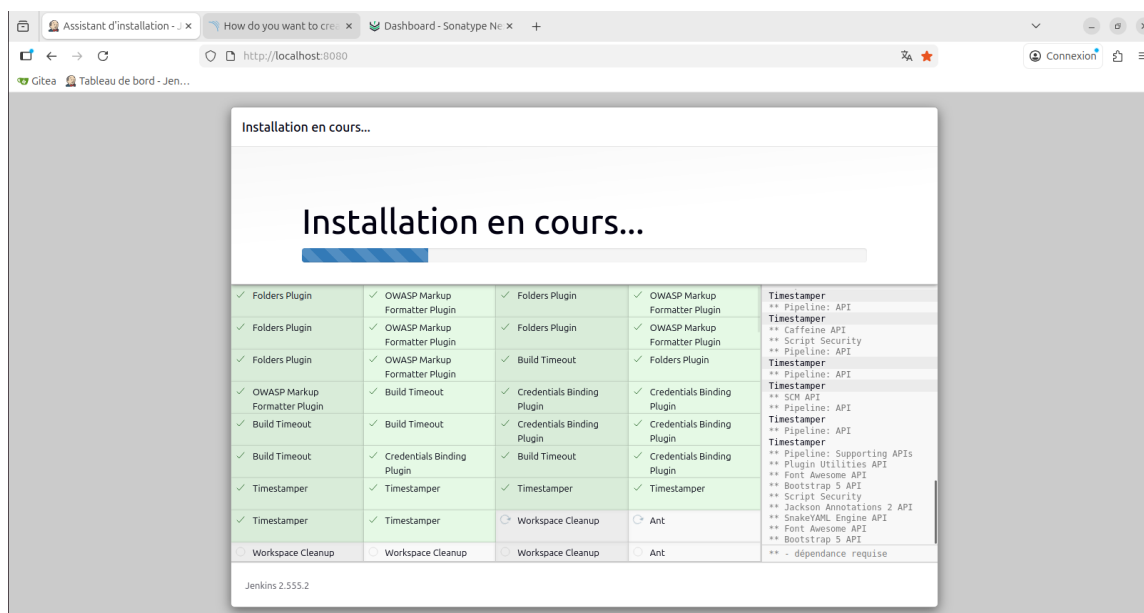


FIGURE 2.6 – Installation des plugins Jenkins

2.4 Conclusion de la semaine 1

À la fin de la première semaine, l'environnement de base a été préparé avec succès. Les conteneurs Docker ont été lancés, Jenkins a été initialisé et les plugins nécessaires ont été installés. Cette phase constitue la base nécessaire pour commencer l'intégration des outils DevOps durant la deuxième semaine.

Chapitre 3

Semaine 2 : Intégration Jenkins, SonarQube et Nexus

3.1 Introduction

La deuxième semaine a été consacrée à l'intégration des outils de la chaîne CI/CD. L'objectif principal était de connecter Jenkins avec SonarQube pour l'analyse de qualité du code, puis avec Nexus pour la gestion des artefacts Maven.

3.2 Travail réalisé par l'Étudiant A

3.2.1 Accès à SonarQube

SonarQube est utilisé dans ce projet pour analyser le code source et détecter les erreurs, les bugs, les vulnérabilités et les mauvaises pratiques de développement. L'accès à son interface web permet de créer un projet, générer un token et consulter les résultats d'analyse.

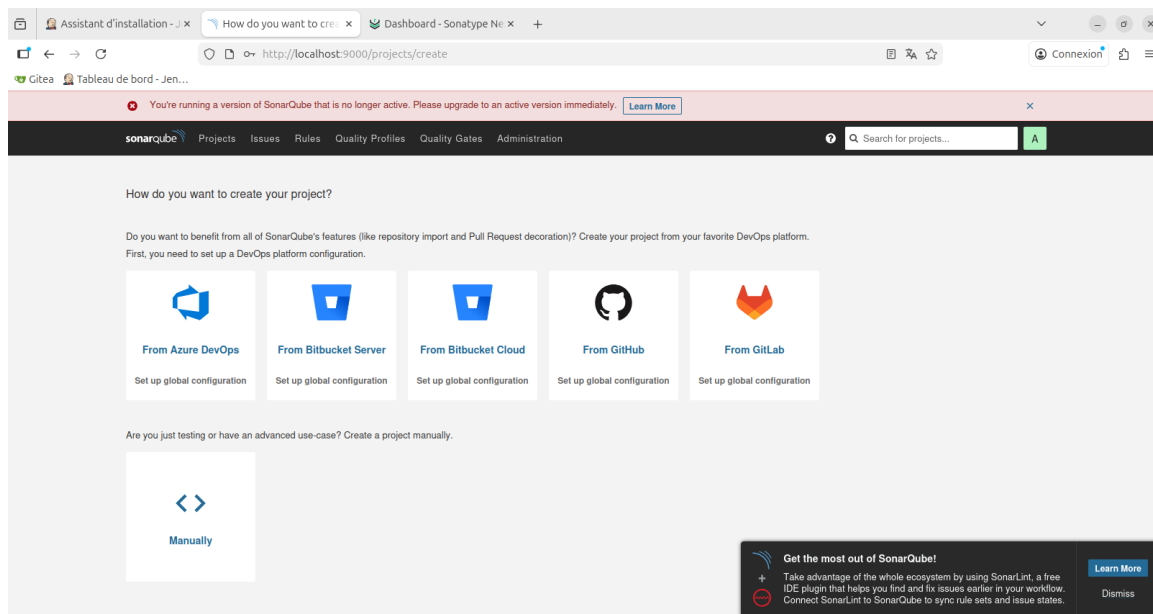


FIGURE 3.1 – Page d'accueil de SonarQube

3.2.2 Affichage du tableau de bord SonarQube

Cette étape montre l'interface de SonarQube après l'accès au tableau de bord. Elle permet de consulter les projets analysés et les informations liées à la qualité du code.

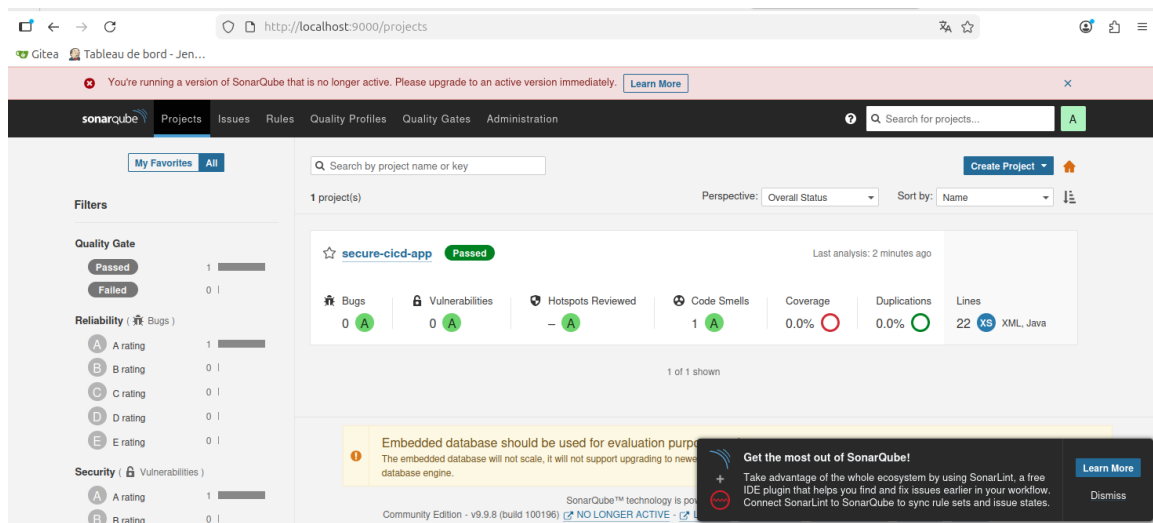


FIGURE 3.2 – Affichage du tableau de bord SonarQube

3.2.3 Création du token Jenkins dans SonarQube

Pour permettre à Jenkins de communiquer avec SonarQube, un token d'accès a été créé. Ce token représente une méthode d'authentification sécurisée entre les deux outils et évite l'utilisation directe du mot de passe administrateur.

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name **Type** **Expires in**

Enter Token Name Select Token Type 30 days Generate

New token "jenkins-token" has been created. Make sure you copy it now, you won't be able to see it again!

Copy `sqa_b5e5519eccb837e4f73382fcc04316507d420831`

Name	Type	Project	Last use	Created	Expiration	
jenkins-token	Global		Never	May 23, 2026	June 22, 2026	Revoke

FIGURE 3.3 – Création du token Jenkins dans SonarQube

3.2.4 Ajout du token SonarQube dans Jenkins

Le token généré dans SonarQube a ensuite été ajouté dans Jenkins au niveau des identifiants globaux. L'identifiant utilisé est `sonar-token`. Il sera appelé dans le fichier `Jenkinsfile` afin d'exécuter l'analyse SonarQube automatiquement pendant le pipeline.

http://localhost:8080/manage/credentials/store/system/domain/_/credential/sonar-token/

Tableau de bord - Jen...

Jenkins Administrer Jenkins Identifiants System Identifiants globaux (illimi... SonarQube token

SonarQube token

SonarQube token

Usage

This credential has not been recorded as used anywhere.
Note: usage tracking requires the cooperation of plugins and consequently...

Update credential

Portée ? Global (Jenkins, agents, items, etc...)

Secret

Concealed Change Password

ID ? sonar-token

Description ? SonarQube token

Sauvegarder

REST API Jenkins 2.555.2

FIGURE 3.4 – Ajout du token SonarQube dans Jenkins

3.2.5 Configuration de SonarQube dans Jenkins

Après l'ajout du token, Jenkins a été configuré pour reconnaître le serveur SonarQube. Cette configuration permet à Jenkins d'envoyer les résultats d'analyse vers SonarQube à chaque exécution du pipeline.

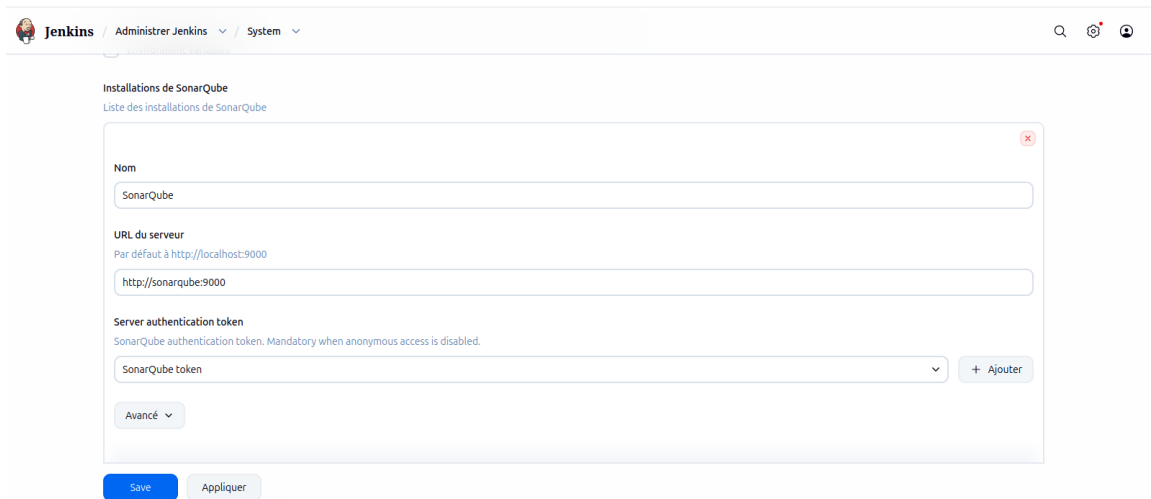


FIGURE 3.5 – Configuration de SonarQube dans Jenkins

3.2.6 Création du fichier Jenkinsfile

Le fichier `Jenkinsfile` décrit les différentes étapes du pipeline CI/CD. Il permet d'automatiser la récupération du code source, la compilation du projet, l'analyse SonarQube et la gestion des artefacts.

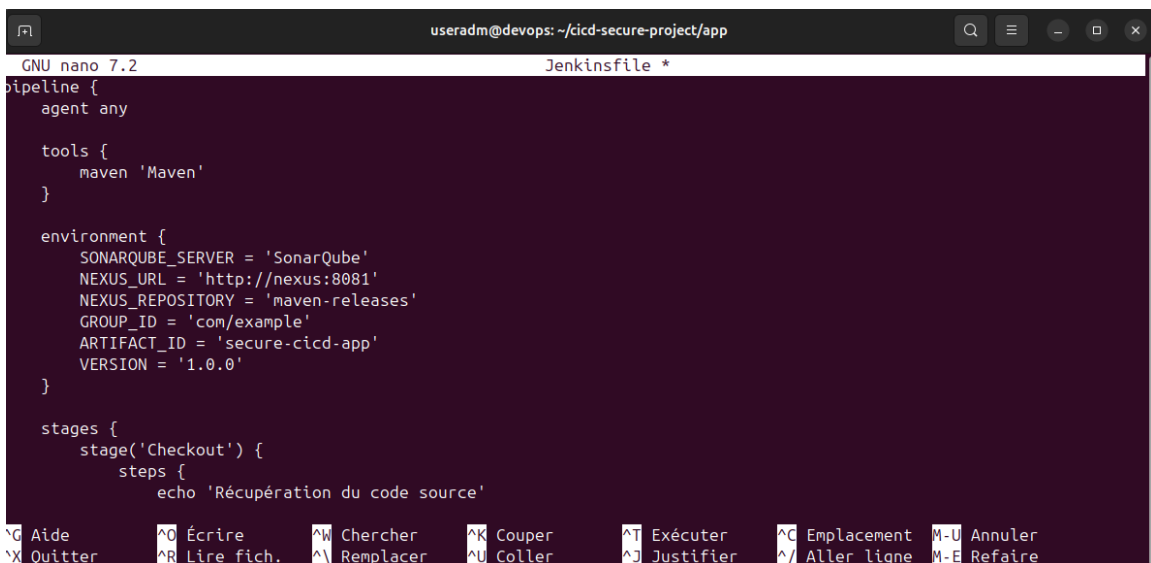


FIGURE 3.6 – Création du fichier Jenkinsfile

3.3 Travail réalisé par l'Étudiant B

3.3.1 Accès à Nexus Repository Manager

Nexus Repository Manager est utilisé pour stocker et gérer les artefacts générés par le pipeline. Dans ce projet, il permet de centraliser les fichiers générés par Maven, comme les fichiers `.jar`.

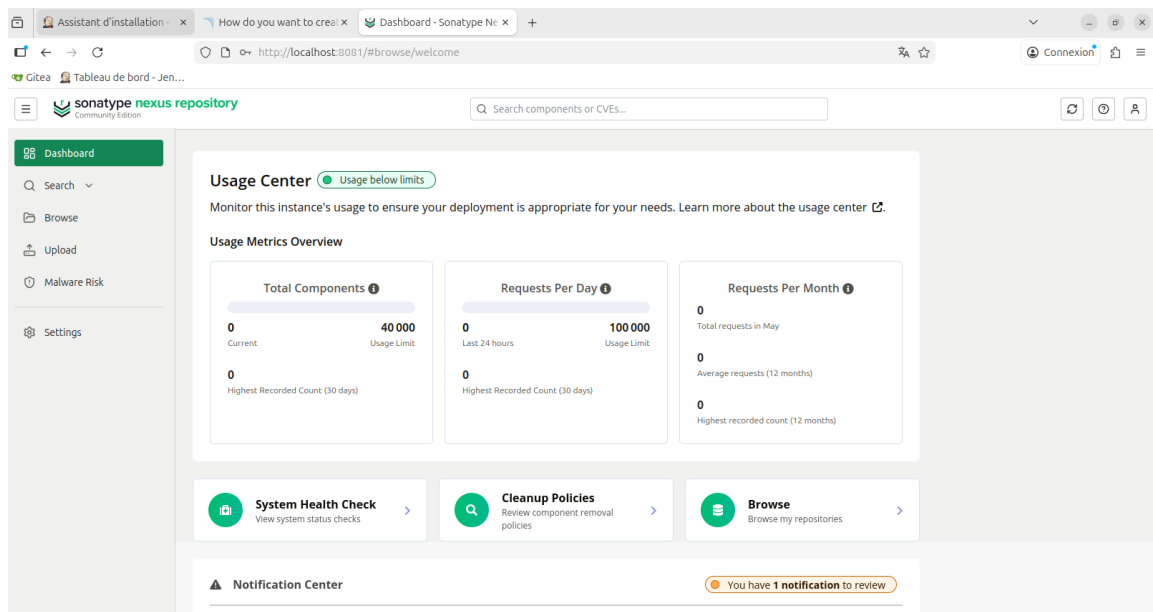


FIGURE 3.7 – Page d’accueil de Nexus Repository Manager

3.3.2 Récupération du mot de passe Nexus

Lors de la première connexion à Nexus, un mot de passe administrateur initial est demandé. Ce mot de passe doit être récupéré afin de finaliser la configuration de Nexus et accéder à son tableau de bord.

```

useradm@devops: ~/cicd-secure-project
: br-9b54344da765: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
link/ether d2:df:b6:5d:e5:6f brd ff:ff:ff:ff:ff:ff
inet 172.18.0.1/16 brd 172.18.255.255 scope global br-9b54344da765
    valid_lft forever preferred_lft forever
inet6 fe80::d0df:b6ff:fe5d:e56f/64 scope link
    valid_lft forever preferred_lft forever
: veth69e0fd8@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-9b54344da765 state UP group default
link/ether 7a:04:be:42:11:e1 brd ff:ff:ff:ff:ff:ff link-netnsid 0
inet6 fe80::7804:beff:fe42:11e1/64 scope link
    valid_lft forever preferred_lft forever
: veth5240af6@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-9b54344da765 state UP group default
link/ether da:7a:a8:32:1b:ee brd ff:ff:ff:ff:ff:ff link-netnsid 2
inet6 fe80::d87a:a8ff:fe32:1bee/64 scope link
    valid_lft forever preferred_lft forever
: veth46f3910@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
link/ether aa:2c:83:d9:57:6f brd ff:ff:ff:ff:ff:ff link-netnsid 1
inet6 fe80::4455:8aff:fe8d:e7a1/64 scope link
    valid_lft forever preferred_lft forever
seradm@devops:~/cicd-secure-project$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
b725dc602f44c29b95b291664b40049
seradm@devops:~/cicd-secure-project$ docker exec nexus cat /nexus-data/admin.password
56beb7b-e908-46eb-96f3-1ef5a37935b8useradm@devops:~/cicd-secure-project$

```

FIGURE 3.8 – Récupération du mot de passe initial de Nexus

3.3.3 Configuration des identifiants Nexus

Cette étape consiste à configurer les identifiants Nexus nécessaires pour permettre à Jenkins d’accéder au repository et d’y publier les artefacts générés par le pipeline.

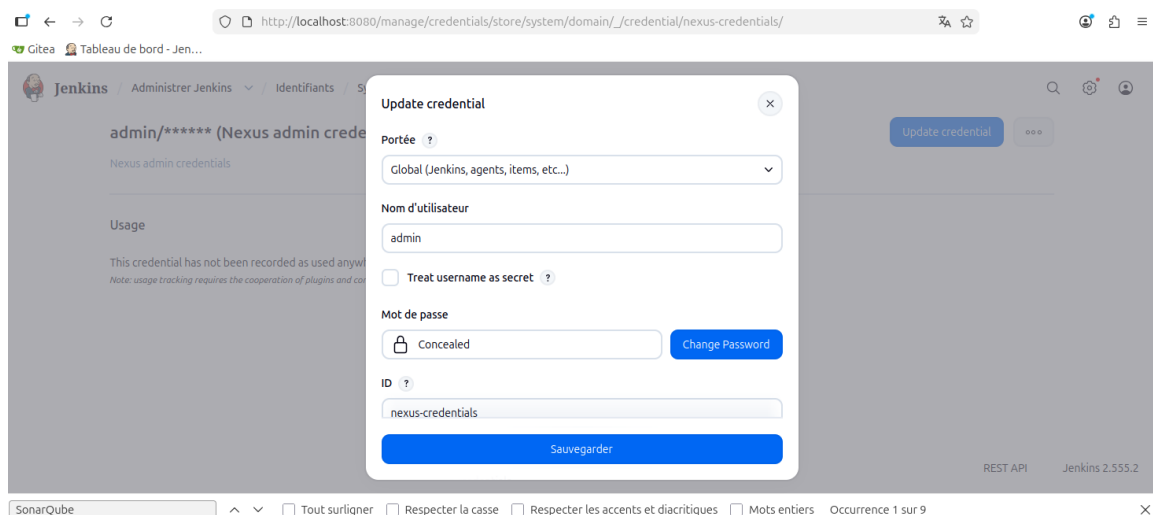


FIGURE 3.9 – Configuration des identifiants Nexus

3.3.4 Consultation des repositories Nexus

Après la connexion à Nexus, les repositories existants peuvent être consultés depuis l'interface d'administration. Ces repositories permettent d'organiser les artefacts selon leur type, leur format et leur usage.

Repositories

[Create repository](#) Filter

	Name ↑	Type	Format	Blob Store	Status	URL	Health check	Firewall Re...	
	maven-central	proxy	maven2	default	Online - Read...		0 0		
	maven-public	group	maven2	default	Online				
	maven-releas...	hosted	maven2	default	Online				
	maven-snaps...	hosted	maven2	default	Online				
	nuget-group	group	nuget	default	Online				
	nuget-hosted	hosted	nuget	default	Online				
	nuget.org-pro...	proxy	nuget	default	Online - Read...		0 0		

FIGURE 3.10 – Consultation des repositories Nexus

3.3.5 Création d'un repository Maven

Un repository Maven a été créé afin de stocker les artefacts générés par le pipeline Jenkins. Cette étape permet de conserver les livrables du projet dans un espace centralisé et réutilisable par les autres membres de l'équipe.

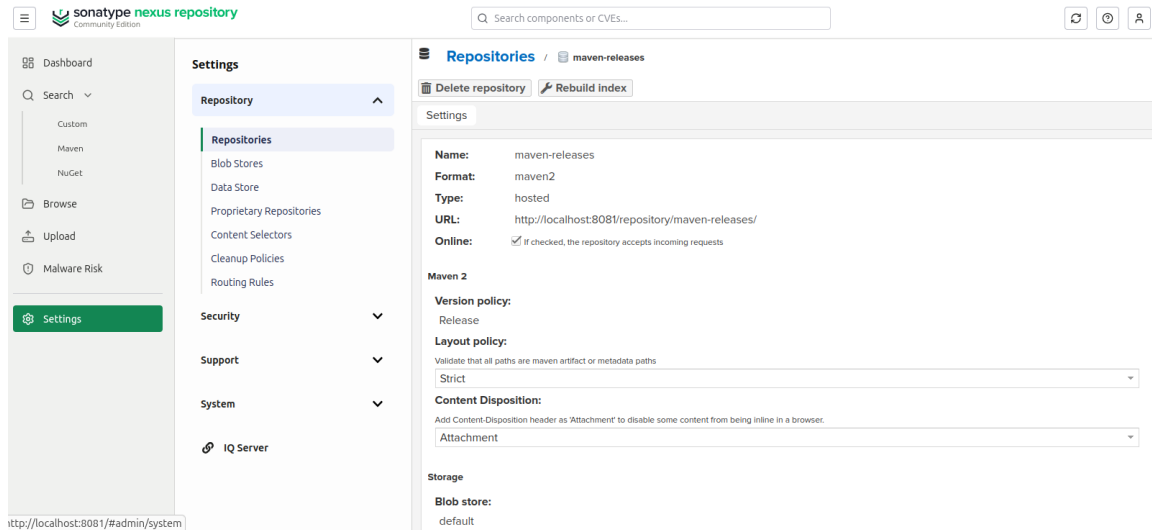


FIGURE 3.11 – Création d'un repository Maven dans Nexus

3.4 Conclusion de la semaine 2

À la fin de la deuxième semaine, les principaux outils de la chaîne CI/CD ont été intégrés. Jenkins a été connecté à SonarQube pour l'analyse du code, et Nexus a été configuré pour la gestion des artefacts Maven. Cette étape permet d'obtenir une chaîne CI/CD plus complète, automatisée et sécurisée.

Chapitre 4

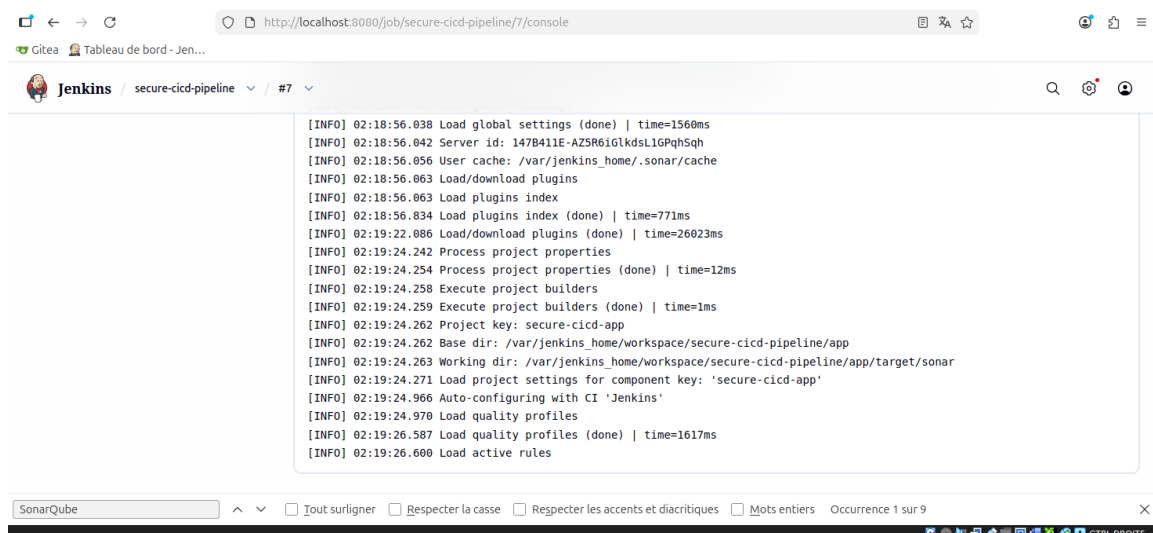
Exécution du pipeline CI/CD

4.1 Introduction

Après la configuration des différents outils, le pipeline Jenkins a été lancé afin de tester l'ensemble de la chaîne CI/CD. Cette étape permet de vérifier que Jenkins peut exécuter correctement les différentes phases définies dans le fichier `Jenkinsfile`.

4.2 Lancement du pipeline Jenkins

Cette étape montre le lancement du pipeline Jenkins après la configuration de Jenkins, SonarQube et Nexus. Elle permet de vérifier le bon fonctionnement de l'orchestration entre les outils.



```
[INFO] 02:18:56.038 Load global settings (done) | time=1560ms
[INFO] 02:18:56.042 Server id: 147B411E-AZ5R6iGkdsL1GPqh5qh
[INFO] 02:18:56.056 User cache: /var/jenkins_home/.sonar/cache
[INFO] 02:18:56.063 Load/download plugins
[INFO] 02:18:56.063 Load plugins index
[INFO] 02:18:56.834 Load plugins index (done) | time=771ms
[INFO] 02:19:22.086 Load/download plugins (done) | time=26023ms
[INFO] 02:19:24.242 Process project properties
[INFO] 02:19:24.254 Process project properties (done) | time=12ms
[INFO] 02:19:24.258 Execute project builders
[INFO] 02:19:24.259 Execute project builders (done) | time=1ms
[INFO] 02:19:24.262 Project key: secure-cicd-app
[INFO] 02:19:24.262 Base dir: /var/jenkins_home/workspace/secure-cicd-pipeline/app
[INFO] 02:19:24.263 Working dir: /var/jenkins_home/workspace/secure-cicd-pipeline/app/target/sonar
[INFO] 02:19:24.271 Load project settings for component key: 'secure-cicd-app'
[INFO] 02:19:24.966 Auto-configuring with CI 'Jenkins'
[INFO] 02:19:24.970 Load quality profiles
[INFO] 02:19:26.587 Load quality profiles (done) | time=1617ms
[INFO] 02:19:26.600 Load active rules
```

FIGURE 4.1 – Lancement du pipeline Jenkins

4.3 Construction du projet avec Maven

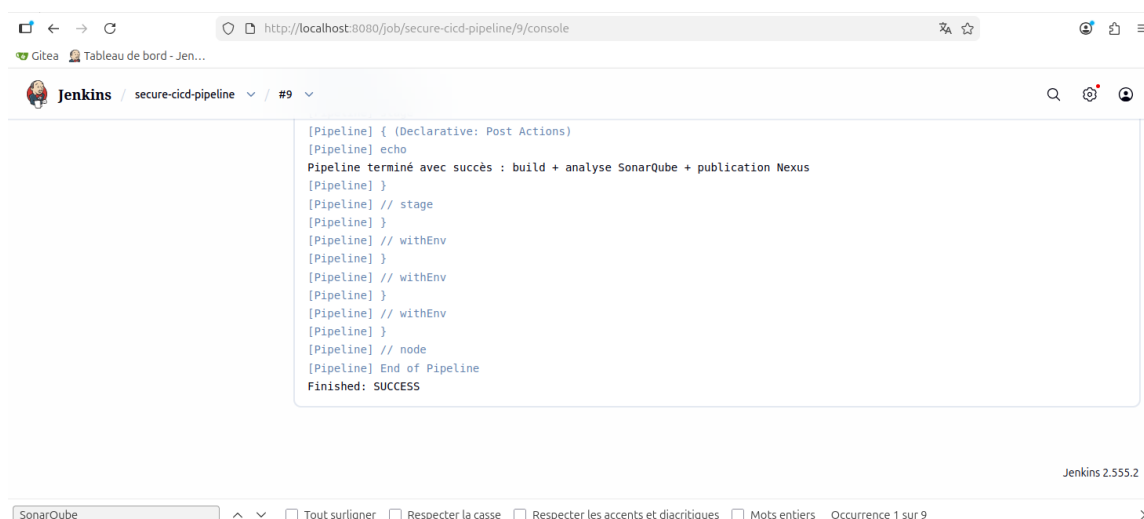
La construction du projet est réalisée avec Maven. Cette étape permet de compiler le code source, d'exécuter les tests et de générer l'artefact final du projet. Elle représente une phase essentielle dans le pipeline CI/CD.

```
INFO] Building jar: /home/useradm/cicd-secure-project/app/target/secure-cicd-app-1.0.0.jar
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 41.388 s
INFO] Finished at: 2026-05-23T01:44:33+02:00
INFO] -----
useradm@devops: ~/cicd-secure-project/app$
```

FIGURE 4.2 – Construction du projet avec Maven

4.4 Succès de l'exécution du pipeline

Cette étape confirme que le pipeline Jenkins s'est exécuté avec succès. Cela signifie que les différentes phases principales du pipeline ont été réalisées correctement.



The screenshot shows the Jenkins web interface at the URL `http://localhost:8080/job/secure-cicd-pipeline/9/console`. The console output displays the following information:

```
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline terminé avec succès : build + analyse SonarQube + publication Nexus
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

At the bottom of the console, the Jenkins version is listed as 2.555.2. Below the console output, there is a search bar with the text "SonarQube" and several checkboxes for search options: "Tout surligner", "Respecter la casse", "Respecter les accents et diacritiques", and "Mots entiers". The search results show "Occurrence 1 sur 9".

FIGURE 4.3 – Exécution réussie du pipeline Jenkins

4.5 Publication de l'artefact dans Nexus

Après la construction du projet Maven, l'artefact généré est publié dans le repository Nexus configuré. Cette étape permet de centraliser les livrables du projet et de faciliter leur réutilisation.

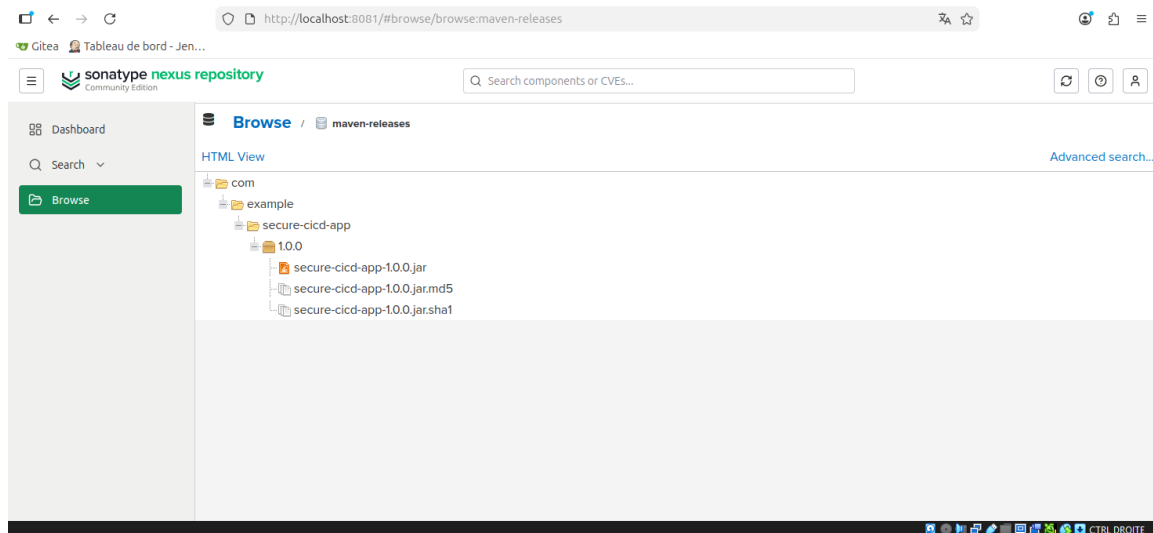


FIGURE 4.4 – Publication de l’artefact dans Nexus

4.6 Résultat obtenu

Le lancement du pipeline confirme que les différents outils sont correctement intégrés. Jenkins joue le rôle d’orchestrateur principal, SonarQube assure l’analyse de qualité du code et Nexus permet la gestion des artefacts générés. Cette automatisation améliore la rapidité, la fiabilité et la sécurité du processus de développement logiciel.

Chapitre 5

Bilan du travail réalisé

5.1 Synthèse des tâches

Le tableau suivant présente une synthèse détaillée des tâches réalisées par chaque étudiant durant les deux semaines du projet.

Période	Étudiant A	Étudiant B
Semaine 1	— Extraction du projet Docker Compose. — Lancement des conteneurs Docker. — Vérification du dépôt Git.	— Accès à Jenkins. — Récupération du mot de passe initial. — Installation des plugins Jenkins.
Semaine 2	— Accès à SonarQube. — Affichage du tableau de bord SonarQube. — Création du token Jenkins. — Ajout du token dans Jenkins. — Configuration de SonarQube dans Jenkins. — Création du Jenkinsfile.	— Accès à Nexus. — Récupération du mot de passe Nexus. — Configuration des identifiants Nexus. — Consultation des repositories. — Création d'un repository Maven. — Test du pipeline avec Maven.

TABLE 5.1 – Synthèse des tâches réalisées par Étudiant A et Étudiant B

5.2 Difficultés rencontrées

Durant la réalisation du projet, plusieurs difficultés peuvent être rencontrées, notamment :

- La configuration des conteneurs Docker et la gestion des ports.
- La récupération des mots de passe initiaux de Jenkins et Nexus.
- L’installation correcte des plugins Jenkins.
- L’intégration du token SonarQube dans Jenkins.
- La création correcte du repository Maven dans Nexus.
- La configuration des identifiants nécessaires à Jenkins.
- La création correcte du fichier **Jenkinsfile**.

5.3 Apports du projet

Ce projet nous a permis de mieux comprendre le fonctionnement d’une chaîne CI/CD moderne. Il nous a également permis de pratiquer l’intégration de plusieurs outils DevOps utilisés dans les environnements professionnels.

Les compétences développées durant ce projet sont :

- La manipulation de Docker et Docker Compose.
- La configuration de Jenkins.
- L’intégration de SonarQube dans un pipeline.
- La gestion des artefacts avec Nexus.
- L’utilisation de Maven dans une chaîne CI/CD.
- La création d’un pipeline automatisé avec un **Jenkinsfile**.
- La sécurisation des accès à travers l’utilisation des credentials Jenkins.

Conclusion générale

Ce projet a permis de mettre en place une chaîne CI/CD sécurisée basée sur Jenkins, SonarQube, Nexus, Docker, Maven et Gitea. La solution réalisée permet d'automatiser plusieurs étapes importantes du cycle de développement logiciel, notamment la compilation du projet, l'analyse de qualité du code et la gestion des artefacts.

La répartition du travail entre Étudiant A et Étudiant B a facilité l'organisation des tâches sur deux semaines. La première semaine a été consacrée à la préparation de l'environnement et à la configuration initiale de Jenkins. La deuxième semaine a permis d'intégrer SonarQube et Nexus dans la chaîne CI/CD.

En conclusion, ce projet représente une base solide pour comprendre les principes de l'intégration continue, de la livraison continue et de la sécurisation du processus de développement logiciel.